Network Intrusion Detection System

A Python-based TCP SYN port scan detector: build, test and results

1. Project Overview

This project is a lightweight intrusion detection system (IDS) written in Python. It reads a network capture file (.pcap), summarizes the traffic by protocol, and raises an alert when it sees behavior that looks like a **TCP SYN port scan**, where one machine probes many ports on another machine to find open services.

Goal: capture network traffic, analyze it automatically, and prove that the detector raises alerts on malicious-looking traffic while staying quiet on normal traffic.

Tools used

Tool	Purpose
Ubuntu on WSL	Linux environment on Windows, where raw packet capture works with proper permissions
tcpdump	Captures live network packets and saves them to a .pcap file
Python 3 + Scapy	Reads the pcap file and lets us inspect (and craft) individual packets
nano	Terminal text editor used to write the scripts
nmap	Port scanner, used on my own machine to generate a real scan for testing

Why WSL instead of VS Code on Windows

I first tried building everything directly in VS Code on Windows, but packet capture needs low-level access to the network interface, which required many extra permissions. Running Ubuntu on WSL solved this, since Linux handles packet capture natively with sudo.

2. Environment Setup (Ubuntu / WSL)

Install the required tools:

```
sudo apt update
sudo apt install tcpdump python3-pip nmap
pip install scapy
```

> If pip refuses with an "externally managed environment" message, use `sudo apt install python3-scapy` instead, or install inside a virtual environment.

3. Capturing Traffic with tcpdump

tcpdump listens on a network interface and writes every packet it sees to a file. The first capture was normal browsing traffic (visiting websites in Chrome):

```
sudo tcpdump -i eth0 -w ~/network_traffic.pcap
# browse some websites, then press Ctrl+C to stop the capture
```

The interface name (eth0 here) can differ between systems. Run `ip link` to list the available interfaces.

- `-i` selects the network interface to listen on.
- `-w` writes the raw packets to a file instead of printing them.
- `sudo` is required because capturing packets needs elevated privileges.

.

4. The Detector: Code Explained Block by Block

The analyzer is `pcap_analyzer.py`. The detection idea is simple: a normal client connects to a few ports on a server, but a scanner sends connection attempts (SYN packets) to *many different ports* on the same target in a short time. If one source IP sends SYNs to at least N distinct ports on the same destination, the script raises an alert (N = 20 by default).

Block 1: Imports

```
import argparse
from collections import Counter, defaultdict
from pathlib import Path

from scapy.all import ICMP, IP, TCP, UDP, rdpcap
from scapy.error import Scapy_Exception
```

- **argparse** lets the script take command-line arguments (the pcap file and an optional threshold).
- **Counter** counts how many packets of each protocol appear; **defaultdict** builds the per-source/destination record of ports without needing to check if a key exists first.
- **Path** gives a clean way to handle and validate the file path.
- **Scapy** provides `rdpcap` (load a capture) and the protocol layers IP, TCP, UDP and ICMP used to inspect each packet.

Block 2: Input validation helper

```
def positive_int(value):
    number = int(value)
    if number < 1:
        raise argparse.ArgumentTypeError("must be at least 1")
    return number
```

This small function makes sure the alert threshold is a whole number of at least 1. If the user types something invalid (like 0 or -5), argparse shows a clear error instead of the program behaving strangely.

Block 3: Command-line arguments

```
parser = argparse.ArgumentParser(
    description="Summarize a PCAP and flag possible TCP SYN port scans."
)
parser.add_argument("pcap", type=Path, help="Path to a .pcap or .pcapng file")
parser.add_argument(
    "--scan-port-threshold",
    type=positive_int,
    default=20,
    help="Distinct destination ports needed to trigger an alert (default: 20)",
)
args = parser.parse_args()
```

Defines how the program is run: `python3 pcap_analyzer.py capture.pcap`, with an optional `--scan-port-threshold` to change the sensitivity. The threshold is the key tuning knob of the whole IDS: lower values catch smaller scans but create more false alarms.

Block 4: Loading the capture safely

```
if not args.pcap.is_file():
```

```
    parser.error(f"PCAP file not found: {args.pcap}")

try:
    packets = rdpcap(str(args.pcap))
except (OSError, Scapy_Exception) as exc:
    parser.exit(1, f"Could not read PCAP file {args.pcap}: {exc}\n")
```

First it checks the file exists, then loads every packet into memory with `rdpcap`. If the file is missing, unreadable or not a valid capture, the program exits with a clear message instead of crashing with a long error trace.

Block 5: Data structures for tracking

```
protocol_counts = Counter()
syn_ports_by_pair = defaultdict(set)
reported_scans = set()
ip_count = 0
```

- **protocol_counts**: running total of TCP, UDP, ICMP and other packets.
- **syn_ports_by_pair**: the core of the detector. For each (source IP, destination IP) pair it stores the *set* of destination ports that received a SYN. A set automatically ignores duplicates, so sending 100 SYNs to port 443 still counts as only 1 port.
- **reported_scans**: remembers which pairs already triggered an alert, so each scan is reported once instead of on every later packet.
- **ip_count**: how many IPv4 packets were analyzed.

Block 6: Looping through packets

```
for packet in packets:
    if IP not in packet:
        continue

    ip_count += 1
    source_ip = packet[IP].src
    destination_ip = packet[IP].dst
```

Each packet is checked one by one. Packets without an IPv4 layer (for example IPv6 or ARP) are skipped. For IPv4 packets it extracts who sent the packet and who it was sent to.

Block 7: Detecting a SYN packet

```
if TCP in packet:
    protocol_counts["TCP"] += 1
    tcp = packet[TCP]
    flags = int(tcp.flags)

    # Count initial SYN packets, not SYN-ACK responses.
    if flags & 0x02 and not flags & 0x10:
```

TCP packets carry flag bits. A connection starts when a client sends a packet with the **SYN** flag (bit 0x02). The server answers with **SYN+ACK** (SYN plus the ACK bit, 0x10). This condition keeps only packets with SYN set and ACK *not* set, meaning real connection attempts. Without this check, a busy server's replies would look like scanning and cause false alerts.

Block 8: Recording ports and raising the alert

```
        pair = (source_ip, destination_ip)
        ports = syn_ports_by_pair[pair]
        ports.add(tcp.dport)

        if (
            len(ports) >= args.scan_port_threshold
            and pair not in reported_scans
        ):
            reported_scans.add(pair)
            print(
                f"ALERT: Possible TCP SYN port scan "
                f"({len(ports)} distinct ports), "
                f"{source_ip} -> {destination_ip}"
            )
```

Every initial SYN adds its destination port to the set for that source/destination pair. As soon as the set reaches the threshold and the pair has not been reported yet, the script prints an ALERT and marks the pair as reported. That is why a 100-port scan prints "20 distinct ports" with the default threshold: the alert fires the moment the count hits 20.

Block 9: Counting other protocols

```
elif UDP in packet:
    protocol_counts["UDP"] += 1
elif ICMP in packet:
    protocol_counts["ICMP"] += 1
else:
    protocol_counts["Other"] += 1
```

Packets that are not TCP are classified as UDP, ICMP or other. This does not affect detection, but gives the traffic summary printed at the end.

Block 10: Summary output and entry point

```
print(f"Read {len(packets)} packets; analyzed {ip_count} IPv4 packets.")
if protocol_counts:
    print("IPv4 protocols: " + ", ".join(
        f"{protocol}={count}"
        for protocol, count in sorted(protocol_counts.items())
    ))
else:
    print("No IPv4 packets found in the capture.")

print("Possible-scan alerts are heuristic, not proof of an attack.")


if __name__ == "__main__":
    main()
```

After the loop, the script prints how many packets were read, the protocol breakdown, and a reminder that alerts are heuristic: a flagged scan is a lead to investigate, not proof of an attack. The last two lines make the file run `main()` only when executed directly, not when imported.

5. Testing the Detector

My first run used only my own Chrome browsing traffic and reported no scans. That is the correct result for normal traffic, but it does not prove the detector works. An IDS must be tested with traffic that *should* trigger it, and with traffic that should *not*.

Step 1: Generate test traffic with Scapy

The script `make_test_pcap.py` crafts four groups of packets and saves them to `test_traffic.pcap`. Because I build the packets myself, I know exactly what the right answer is.

```
from scapy.all import Ether, IP, TCP, wrpcap

# Fixed MAC addresses avoid Scapy's "getmacbyip" warnings for fake IPs
eth = Ether(src="00:11:22:33:44:55", dst="66:77:88:99:aa:bb")

packets = []

# 1. SHOULD ALERT: 10.0.0.5 SYN-scans ports 1-100 on 10.0.0.10
for port in range(1, 101):
    packets.append(eth/IP(src="10.0.0.5", dst="10.0.0.10")
                   /TCP(sport=40000, dport=port, flags="S"))

# 2. should NOT alert: only 10 ports (below the threshold of 20)
for port in range(1, 11):
    packets.append(eth/IP(src="10.0.0.6", dst="10.0.0.10")
                   /TCP(sport=41000, dport=port, flags="S"))

# 3. should NOT alert: 50 SYN-ACK replies (a server answering, not scanning)
for port in range(1000, 1050):
    packets.append(eth/IP(src="10.0.0.20", dst="10.0.0.5")
                   /TCP(sport=port, dport=50000, flags="SA"))

# 4. should NOT alert: many SYNs, but all to port 443 (normal browsing)
for i in range(50):
    packets.append(eth/IP(src="10.0.0.7", dst="10.0.0.30")
                   /TCP(sport=42000 + i, dport=443, flags="S"))

wrpcap("test_traffic.pcap", packets)
print(f"Wrote {len(packets)} packets")
```

Case	Traffic	Expected	Why
1	10.0.0.5 sends SYNs to 100 different ports	ALERT	Classic port scan
2	10.0.0.6 sends SYNs to 10 ports	No alert	Below the threshold of 20
3	10.0.0.20 sends 50 SYN-ACK packets	No alert	Replies from a server, not connection attempts
4	10.0.0.7 sends 50 SYNs, all to port 443	No alert	Many connections to one port is normal

Step 2: Run the detector on the test file

```
python3 make_test_pcap.py
python3 ~/pcap_analyzer.py test_traffic.pcap
```

Result:

```
ALERT: Possible TCP SYN port scan (20 distinct ports), 10.0.0.5 -> 10.0.0.10
Read 210 packets; analyzed 210 IPv4 packets.
IPv4 protocols: TCP=210
Possible-scan alerts are heuristic, not proof of an attack.
```

Exactly one alert, for the real scan only. This is one **true positive** and three **true negatives**: the detector caught the scan and stayed quiet on the other three cases.

Step 3: Change the threshold to show sensitivity

```
python3 ~/pcap_analyzer.py test_traffic.pcap --scan-port-threshold 5

ALERT: Possible TCP SYN port scan (5 distinct ports), 10.0.0.5 -> 10.0.0.10
ALERT: Possible TCP SYN port scan (5 distinct ports), 10.0.0.6 -> 10.0.0.10
Read 210 packets; analyzed 210 IPv4 packets.
IPv4 protocols: TCP=210
Possible-scan alerts are heuristic, not proof of an attack.
```

With a threshold of 5, the smaller 10-port probe from 10.0.0.6 is also detected. This shows the core trade-off in any IDS:

Threshold	Effect
Low (e.g. 5)	Catches small, quiet scans, but more risk of **false positives** on legitimate traffic
High (e.g. 20+)	Fewer false alarms, but small scans can slip through as **false negatives**

Step 4: Test against a real scan with nmap (own machine only)

To confirm the detector also works on a real tool, not only crafted packets, capture an nmap scan against my own WSL machine. Only scan systems you own.

```
# Terminal 1: capture on the loopback interface
sudo tcpdump -i lo -w real_scan.pcap

# Terminal 2: scan my own machine (SYN scan of ports 1-200)
sudo nmap -sS -p 1-200 127.0.0.1

# Back in Terminal 1: press Ctrl+C, then analyze
python3 ~/pcap_analyzer.py real_scan.pcap
```

6. Quick Command Reference

Task	Command
Install tools	sudo apt install tcpdump python3-pip nmap | pip install scapy
Capture traffic	sudo tcpdump -i eth0 -w capture.pcap (Ctrl+C to stop)
Create test pcap	python3 make_test_pcap.py
Run the IDS	python3 ~/pcap_analyzer.py capture.pcap
Change sensitivity	python3 ~/pcap_analyzer.py capture.pcap --scan-port-threshold 5

Task	Command
Real scan test	sudo nmap -sS -p 1-200 127.0.0.1 (while tcpdump -i lo runs)

7. Limitations and Future Improvements

- **No time window:** ports are counted over the entire capture, so a long capture of normal traffic could false-alarm, while real scans happen within seconds. Next step: alert on N ports within a short window (for example 10 seconds) using the packet timestamps.
- **IPv4 only:** IPv6 packets are skipped, and modern browsers use IPv6 often.
- **SYN scans only:** it does not detect FIN, NULL or Xmas scans, UDP scans, scans of one port across many hosts, or SYN floods.
- **Offline analysis:** it reads a finished pcap and does not monitor live traffic. It also loads the whole file into memory, which limits the size of capture it can handle.
- **Heuristic detection:** an alert means the behavior looks like a scan; a human must investigate before concluding there was an attack.

Planned next features: time-window detection, a SYN flood rule (many SYNs to the same port in a short time), and test packets for each new rule.